certora

Security Assessment & Formal Verification Final Report



Reserve Folio Rebalancing

April 2026

Prepared for Reserve

certora

Table of Contents

Project Summary **4**
Project Scope 4
Project Overview 4
Protocol Overview 4
Assessment Methodology **5**
Threat and Security Overview **6**
System overview 6
1. Protocol, Rebalance and auction consistency 6
2. Trusted filler risks 7
3. Privileged role misuse risks 8
4. Underlying token risks 8
Findings Summary 10
Severity Matrix 10
Detailed Findings **11**
Medium Severity Issues 13
M-01: REBALANCE_MANAGER can allow trustedFillers to rugpull the folio 13
M-02: REBALANCE_MANAGER can allow the AUCTION_LAUNCHER to rugpull the folio 14
M-03: DEFAULT_ADMIN_ROLE can mint arbitrary shares 15
Low Severity Issues 16
L-01: Low asset to share ratio may result in users losing non-negligible value 16
L-02: REBALANCE_MANAGER can add ad-hoc tokens to grief the protocol when tradeAllowlistEnabled=false 17
L-03: DAO and folio fees both set to 0 can DOS the protocol 18
L-04: TVL fees are applied retroactively, and can be avoided 19
Informational Severity Issues 20
I-01: Potential fee optimizations 20
I-02: Reliance on balanceOf() leaves the protocol open to donations 21
I-03: Auctions can have overlapping times 22
I-04: Risks associated with malicious or exploited trustedFillers 23
I-05: AUCTION_LAUNCHER can bypass the weight.spot=0 restriction on the REBALANCE_MANAGER 25
I-06: If no shares remain, mint() cannot be called 26
I-07: addToBasket() and removeFromBasket() may grief redeem() and mint() calls 27
I-08: Closing trusted filler does not automatically remove assets from basket 28
I-09: Setting the folio as a fee recipient DOSs fee distribution 29

I-10: Trusted fill flow can break the rebalance.details[].maxAuctionSize cap ... 30
Formal Verification ... 31
Verification Methodology ... 31
Verification Notations ... 32
General Assumptions and Simplifications ... 32
Formal Verification Properties Overview ... 34
Detailed Properties ... 36
P-01. When not in an auction, token/share ratio should not decrease, for each underlying token, up to fees ... 36
P-02. When in auction, the total value of each share should not decrease ... 36
P-03. For each token, in each auction, the auction limit (rebalance.details[].maxAuctionSize) is not exceeded ... 38
P-04. Bid flow is equivalent to trusted fill flow ... 39
P-05. When auction is opened, we only trade tokens outside the auction's range ... 40
P-06. _price() monotonically decreases throughout an auction ... 40
P-07. Splitting bids is equivalent to a large bid, up to rounding ... 41
P-08. Share quantities remain the same unless mint or redeem are called or fees are recalculated/distributed (e.g. during totalSupply()) ... 42
P-09. We can sell tokens via bid if and only if they are in surplus (above weight.high). We can buy tokens via bid if and only if they are in deficit (below weight.low). We cannot create deficit nor surplus via bid ... 42
P-10. If a token is removed from the basket, the current balance of it is 0, or the user is admin ... 43
Disclaimer ... 44
About Certora ... 44

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Reserve Folio Rebalancing	https://github.com/reserve-protocol/reserve-index-dtf/blob/main/contracts/Folio.sol	dc99d3245ee1c427974536fbb5f94e14843f3134	Solidity

Project Overview

This document describes the security review of **Reserve Folio Rebalancing**. The work was undertaken from **March 16th, 2026** to **April 20th, 2026**.

The following contract list is included in our scope:

```
contracts/Folio.sol
contracts/utils/RebalancingLib.sol
```

The Certora Prover demonstrated that the implementation of the Solidity contracts above is correct with respect to the formal rules written by the Certora team. In addition, the team performed a manual audit of all the Solidity contracts**.** During the verification process and the manual audit, the Certora team discovered bugs in the Solidity contracts code, as listed on the following page.

Protocol Overview

Reserve Folio allows on-chain creation and management of asset portfolios. It also allows rebalancing these portfolios to match desired portfolio profiles. This enables the creation and management of DTFs (Decentralized Token Folios), a DeFi on-chain replacement to ETFs.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

System overview

The folio has multiple underlying assets backing its shares. The assets backing the folio are tracked by a basket. There are multiple permissioned actors: the DEFAULT_ADMIN_ROLE, the REBALANCE_MANAGER, the AUCTION_LAUNCHER, and implicitly the trustedFillerRegistry's owner.

The DEFAULT_ADMIN_ROLE sets parameters for rebalances (such as price and weight controls for the AUCTION_LAUNCHER). The REBALANCE_MANAGER sets rebalance parameters and starts rebalances, and AUCTION_LAUNCHER start auctions within a rebalance, with a failsafe unrestricted auction. All actors can close rebalances and auctions, to allow multiple actors to stop any misbehaving actor. This results in granular control over the system, with most roles being limited in the extent of damage they can cause.

The threats have been split into 4 categories: Protocol, rebalance and auction consistency, Trusted filler risks, Privileged role misuse risks, and Underlying token risks.

Note: We did not report as issues already documented threats, but did include them in the threat model. This includes REBALANCE_MANAGER being able to grief the protocol, and AUCTION_LAUNCHER being able to profit within the margins defined by REBALANCE_MANAGER on ATOMIC_SWAP, etc. We did include in the report any undocumented effects that are caused by documented threats.

1. Protocol, Rebalance and auction consistency

Risk description: The folio allows rebalancing according to parameters set by the privileged roles (DEFAULT_ADMIN_ROLE, REBALANCE_MANAGER and AUCTION_LAUNCHER). If the protocol state is inconsistent, it could propagate to users trading unintended assets at unintended prices.

Key considerations include:

- **Multiple open rebalances:** If multiple rebalances remain open, AUCTION_LAUNCHER may be able to launch an auction with old parameters.
- **Multiple open auctions:** If multiple auctions remain open, users may be able to bid with old auction parameters.
- **Proper auction closure:** If an auction remains open, users may be able to continue bidding on it beyond the allocated timeframe.
- **Parameter enforcement:** If REBALANCE_MANAGER can pass DEFAULT_ADMIN_ROLE restrictions (price/weight control, bids enabled, auction length), or AUCTION_LAUNCHER

bypasses REBALANCE_MANAGER limits, auctions could proceed with risky parameters enabling problematic prices or other bad parameters, resulting in loss of user value.

- **Asset removal during auction:** If an auction remains open, users may be able to continue bidding on it beyond the allocated timeframe.
- **Parameter changes auction:** If the weight or price controls change during a rebalance, or the bids enabling/disabling changes immediately, it could affect user risks or allow a privileged role to take advantage of the system.
- **Reentrancy protection:** If the reentrancy protections are not properly implemented, the system might rely on an inconsistent state.

Relevant rules:

- P-02, P-03 - These properties enforce the ongoing auction limits. This guarantees that even if some old auction/rebalance is active, the current auction's limits are enforced on it.
- P-04 - Guarantees share consistency - verifies share number can only change through the correct paths.

Impact: Potentially allowing bad trades, resulting in loss of user value.

2. Trusted filler risks

Risk description: Trusted fillers are external contracts, which can create bids and fill them at a later point. The current implementation relies on CowSpaw, but may change in the future. The trusted fillers are not currently checked, and are assumed to be correct - giving them substantial control over allocated assets.

Key considerations include:

- **Trusted filler reverting:** Most functions in the folio attempt to close the open trusted filler if one exists. A trusted filler reverting may DOS the protocol.
- **Malicious registry:** The trusted fillers are opened through a registry. A malicious or exploited registry may open trusted fillers which can take the allowance allocated by the folio without giving any assets back.
- **Bad trades:** The trusted fillers may trade at rates below the required rate given to them when opened, resulting in less bought tokens being returned than the amount required.

Relevant rules: No rules were created, as the trusted fillers were assumed to be correct, and a mock trusted filler was used instead.

Impact: May DOS the protocol (rendering user funds being inaccessible), steal a portion of user funds, or make bad trades hurting user value.

3. Privileged role misuse risks

Risk description: The many trusted roles in the system (DEFAULT_ADMIN_ROLE, REBALANCE_MANAGER and AUCTION_LAUNCHER) have substantial power over it. Additionally the TrustedFillerRegistry is implicitly trusted (see above). These roles can misuse their power to create bid options favorable to themselves, and can lock in user funds.

Key considerations include:

- **Privileged roles acting together:** While each role is designed to have limited power over the folio, when acting together each role may bypass restriction on a different role, allowing full misuse of the system.
- **Weight and price manipulation:** The ability to set both weights and prices together allows REBALANCE_MANAGER to sell arbitrary assets in the bucket cheaply, and to buy arbitrary assets in the bucket expensively, resulting in user losses. AUCTION_LAUNCHER may also do this, within the limits defined by REBALANCE_MANAGER.
- **Asset additions:** DEFAULT_ADMIN_ROLE, and without an allowlist REBALANCE_MANAGER, may add malicious tokens, exploiting underlying token risks.
- **Asset removals:** DEFAULT_ADMIN_ROLE, can remove a token from the basket, resulting in loss of user value.

Relevant rules:

- P-02, P-03 - These properties enforce the ongoing auction limits, guaranteeing that once set, they cannot be expanded while the auction is ongoing.

Impact: Can result in rugpulls by privileged users, protocol griefing, and bad trades resulting in loss of user value.

4. Underlying token risks

Risk description: Some tokens have non-standard behaviours, and may behave differently than expected. Additionally, having varying asset to share ratios result in potential risks both from low and high ratios. Donations may also manipulate asset ratios.

- **Underlying token behaviour mapping:** If an underlying token behaves unexpectedly, this may result in a DOS to the folio (for example pausable tokens), in wrong accounting (for example fee on transfer tokens), or other risks (for example tokens with TVL fees, such as other folios). An additional risk are low liquidity tokens, where users do not have enough liquidity to mint shares. These effects may DOS the system or hurt user value.

- **Low and high asset to share ratios:** Low asset to share values may result in users needing to pay 1 wei on mints while getting nothing from redeemed, and high asset to share ratios may result in users requiring high liquidity of some asset to get a single share, limiting investability in the folio.
- **Asset price swings:** Price swings in the underlying assets may make rebalance and auction parameters obsolete, allowing external users to profit off folio users.
- **Donations:** Donations may inflate asset to share ratios, making users unable to mint folio shares. Donations may also make the folio unaligned with the intended spread of underlying assets, shifting risk profiles.

Relevant rules:

- P-01 - Assuming standard ERC-20 underlying tokens, the token to share ratio does not decrease, except for specific defined paths (bids, fees, removal from basket).

Impact: Can result in partial or complete DOS of the protocol, and loss of user value through trades based on stale data.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	3	3	-
Low	4	3	1
Informational	10	10	3
Total	17	16	4

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
M-01	REBALANCE_MANAGER can allow trustedFillers to rugpull the folio	Medium	Acknowledged
M-02	REBALANCE_MANAGER can allow the AUCTION_LAUNCHER to rugpull the folio	Medium	Acknowledged
M-03	DEFAULT_ADMIN_ROLE can mint arbitrary shares	Medium	Acknowledged
L-01	Low asset to share ratio may result in users losing non-negligible value	Low	Acknowledged
L-02	REBALANCE_MANAGER can add ad-hoc tokens to grief the protocol when tradeAllowlistEnabled=false	Low	Acknowledged
L-03	DAO and folio fees both set to 0 can DOS the protocol	Low	Fixed
L-04	TVL fees are applied retroactively, and can be avoided	Low	Acknowledged
I-01	Potential fee optimizations	Informational	Acknowledged
I-02	Reliance on balanceOf() leave the protocol open to donations	Informational	Acknowledged
I-03	Auctions can have overlapping times	Informational	Acknowledged
I-04	Risks associated with malicious or exploited trustedFillers	Informational	Acknowledged

I-05	AUCTION_LAUNCHER can bypass the weight.spot=0 restriction on the REBALANCE_MANAGER	Informational	Fixed
I-06	If no shares remain, mint() cannot be called	Informational	Acknowledged
I-07	addToBasket() and removeFromBasket() may grief redeem() and mint() calls	Informational	Acknowledged
I-08	Closing trusted filler does not automatically remove assets from basket	Informational	Fixed
I-09	Setting the folio as a fee recipient DOSs fee distribution	Informational	Fixed
I-10	Trusted fill flow can break the rebalance.details[].maxAuctionSize cap	Informational	Acknowledged

Medium Severity Issues

M-01: REBALANCE_MANAGER can allow trustedFillers to rugpull the folio

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/Folio.sol	Status: Acknowledged	

Description:

The REBALANCE_MANAGER can set the desired weight of all assets in the folio to be 0, expect one, set high. These assets can then be approved to trustedFillers based on the bids against this asset. The trustedFillers can then take these assets without returning them. This allows the trustedFillers to take all assets except for one type.
In order to take the entire amount, an additional asset may be left with a low weight, then used as the primary asset to drain the original primary asset, leaving only dust amounts of it.

Recommendations:

Consider adding the trustedFiller protection specified in I-04.
Consider emitting an event and adding time delays in cases where there are substantial limit, weight or price differences between the previous rebalance and the new rebalance.

Customer Response: Acknowledged as a limitation. The trusted filler circuit breaker described in I-04 was reverted, as protecting against a malicious or compromised trusted filler would introduce additional complexity and potential failure modes. As an emergency mitigation, the AUCTION_LAUNCHER can call endRebalance() to close any ongoing rebalance.

M-02: REBALANCE_MANAGER can allow the AUCTION_LAUNCHER to rugpull the folio

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/Folio.sol	Status: Acknowledged	

Description:

The REBALANCE_MANAGER can set the desired weight of all assets in the folio to be 0, except one, set high. This asset can then be set with a high price, and all other prices low. The price control needs to be set to ATOMIC_SWAP, bypassing the auction warmup period. The AUCTION_LAUNCHER can then sell little of the chosen asset in return for the entire amount of other assets. This allows the auction launcher to take all assets except for one type.
In order to take the entire amount, an additional asset may be left with a low weight, then used as the primary asset to drain the original primary asset, leaving only dust amounts of it.

Recommendations:

Consider adding time delays before opening ATOMIC_SWAP auctions.
Consider emitting an event and adding time delays in cases where there are substantial limit, weight or price differences between the previous rebalance and the new rebalance.

Customer Response: Acknowledged. Known and documented behavior under ATOMIC_SWAPs.

M-03: DEFAULT_ADMIN_ROLE can mint arbitrary shares

Severity: Medium	Impact: High	Likelihood: Low
Files: contracts/Folio.sol	Status: Acknowledged	

Description:
The DEFAULT_ADMIN_ROLE can remove any assets. When no assets are in the pool, shares can be minted without requiring any assets.
So, the DEFAULT_ADMIN_ROLE can remove all assets, mint arbitrary shares, and add these assets back.

Recommendations:
Enforce that basket.length is not 0, or is larger than some minimum, on mint and redeem.
Consider preventing mints in the same block an asset is removed, make the party profiting from "cheap" shares be indeterminate.

Customer Response: Acknowledged.

Low Severity Issues

L-01: Low asset to share ratio may result in users losing non-negligible value

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/Folio.sol	Status: Acknowledged	

Description:

If the amount of shares is substantially larger than the amount of asset, for some asset, the rounding from redeeming will be, in most cases, round down to 0. This means that most users will deposit 1 wei of the asset and receive 0 back on redeem, leaving this as profit to large users.
For example, assuming 5 mints per minute, of small amount, and no large mints/redeems, in 1 day the folio will gain 5*60*24=7200 wei. A large user holding 5% of the shares, will gain 360 wei of this. While this is usually a small amount, this might be substantial with some assets: for example, with BTC trading at 75,000 USD, with 6 decimals, each user will lose 7.5 US cents, while the large holders will gain 7200 wei worth 540 USD.
There are multiple paths for this situation to happen:

- Low deposit of some asset on init.
- REBALANCE_MANAGER setting an asset to a low weight, allowing users to trade it down.
- A donation or small sell of a newly added asset.

Recommendations:

Consider this implication when choosing assets, and especially new assets, to a folio.
Consider enforcing a minimum share to asset ratio (even 1 to 1000 will limit the loss on most trades), both on init and when trading (i.e, a trade will either leave enough of the relevant asset or take all of it).

Customer Response: Acknowledged.

L-02: REBALANCE_MANAGER can add ad-hoc tokens to grief the protocol when tradeAllowlistEnabled=false

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/Folio.sol	Status: Acknowledged	

Description:

When tradeAllowlistEnabled=false, the REBALANCE_MANAGER can add any tokens they want to the basket by specifying them in the rebalance. They can use this to add non-functioning tokens to grief the system, or add dummy tokens with high weights and prices, making them the only ones who can trade them. The AUCTION_LAUNCHER can avoid adding them to auctions, but they will still be added in the unrestricted auctions. (The AUCTION_LAUNCHER can also cooperate with the REBALANCE_MANAGER to exploit this.)

Recommendations:

Consider documenting this behaviour as a known risk.
Consider making the allowList mandatory, or adding an option to prevent the REBALANCE_MANAGER from adding tokens not already in the basket.

Customer Response: Acknowledged.

L-03: DAO and folio fees both set to 0 can DOS the protocol

Severity: Low	Impact: Medium	Likelihood: Low
Files: contracts/utils/FolioLib.sol	Status: Fixed	

Description:
When calculating fee accumulation, a correction is calculated (FolioLib.sol line 84). To do this, the DAO fee floor is divided by the maximum between the folio TVL fee and the DAO fee floor. If both the folio fee, and the DAO fee floor are 0, this will result in a division by 0, DOSing the folio.

Recommendations:
If both the folio TVL fee and the DAO fee floor are 0, return at the start of the function (after getting the DAO fee floor).

Customer Response: Mitigated with div-by-zero protection in 2d897.

Fix Review: Fixed.

L-04: TVL fees are applied retroactively, and can be avoided

Severity: Low	Impact: Low	Likelihood: Medium
Files: contracts/Folio.sol	Status: Acknowledged	

Description:
Since TVL fees are calculated once a day (Folio.sol line 1015), users who mint in between the updates are paying fees for the time between the last accumulation and the time they minted. Similarly, users redeeming right before a fee update avoid being charged for the time between the last update and the redeem.

Recommendations:
Consider calculating the fees more often (or every time) to minimize the times users can cheat the protocol. Consider setting these times after a comparison between fees avoided and gas prices of redeeming, waiting a block, minting.

Customer Response: Acknowledged. Worth noting though there is no MEV opportunity possible around the discrete fee boundaries, which is really what the primary concern with discrete boundaries is. Unfairness within the day is therefore not really an issue because it's not rational to try to time your mint/redeems around the day boundary.

Why: The minimum mint fee of 3 bps in combination with the maximum TVL fee of 10% annually makes it always unprofitable to perform MEV attacks around the 24h fee boundary.

Informational Severity Issues

I-01: Potential fee optimizations

Files:

contracts/utils/FolioLib.sol

Description:

1. Taking folioFeeForSelf form TVL fees recycles shares

When taking the folioFeeForSelf from the TVL fees, we are taking fees from users and not minting them, returning the value to these same users. This results in no real profit to users but takes some gas.

2. In TVL fee calculations, the `feeFloor` exponent can be calculated in advance.

The feeFloor is calculated from the daoFeeFloor each time computeFeeShares is called. This calculation requires taking an exponent. While this relies on an external variable, this calculation can be done when updating this variable and saved in the external contract, instead of each time the computeFeeShares is called.

Recommendations:

1. Consider not taking the folioFeeForSelf fee, instead calculating this once into the fees, and accounting for this difference when configuring the TVL fees.
2. Consider doing this calculation when updating the daoFeeFloor in the external contract, and adding this variable to daoFeeRegistry.getFeeDetails().

Customer Response: Acknowledged.

I-02: Reliance on balanceOf() leaves the protocol open to donations

Files:

contracts/Folio.sol

Description:

The reliance on balanceOf() leaves the program open to donation attacks. While we have not found realistic exploit scenarios based on this, there may exist unconsidered scenarios for such donation attacks.

Recommendations:

Consider using virtual balances to keep track of underlying token amounts. If this is implemented, consider also adding an "integration" mechanic for absorbing donations to the folio, or a "rescue" mechanic (with rugpull protections to prevent the admin from removing an asset from the basket, then rescuing it).
If adding virtual balances, consider also verifying that balanceOf() >= virtual amount, for all tokens.

Customer Response: Acknowledged.

I-03: Auctions can have overlapping times

Files:

contracts/Folio.sol

Description:

When a rebalance ends, the current auction may continue. When a new rebalance begins, no check is done to the current auction. When an auction is then opened in the new rebalance, it closes the previous auction **only if it was within this rebalance**. This means that the new auction can be opened while the previous auction's time has not elapsed.
Note that since _price() checks that bids were done in the correct rebalance, the old auction does not enable trades, but this state is unintended, and may confuse integrators.

Recommendations:

Consider closing previous auctions when opening new ones, even if they are from a previous rebalance. Alternatively, consider adding a “isAuctionOpen” function that will include both the time and rebalance nonce checks, to make sure the old auctions are explicitly closed.

Customer Response: Acknowledged.

I-04: Risks associated with malicious or exploited trustedFillers

Files:
contracts/Folio.sol contracts/utils/RebalancingLib.sol

Description:
The trustedFillers are explicitly trusted entities in the system, but are external to the folio. Specifically, they are assumed to never revert after the block they were created, to always return unused funds and to trade used funds at at least the specified price.
When working other than intended, they can:

- DOS the folio by reverting on all relevant functions.
- Take tokens approved to them with returning any tokens.
  - By reusing this, all surpluses of all tokens in an auction can be stolen.
- Trade unprofitably.

This also results in the trustedFillerRegistry's owner being a permissioned folio actor.
The trust assumption on trustedFolios can be enforced and checked, preventing these potential problems.
Note: This is marked as informational as these problems are explicitly documented.

Recommendations:
In addition to getting the balance difference before and after the call to activeTrustedFill.closeFiller(), we can also check this difference: We can check that we received enough tokens back, meaning what we sold, was at the right price, and what wasn't sold was returned (something like bought*requested_price_in_sold >= sold = allowed_sold - returned_sold).
This might require saving some values on the folio side instead of the filler side, as these values are currently taken from the filler.
Additionally, a try-catch can be added to prevent a DOS from the trustedFiller.
If either of these break, we can deactivate trusted filler creation (trustedFillerEnabled = false;) to limit the losses to the maximum traded in an auction of the sold token. This can be limited to a lower amount by adding a separate limit to the maximal assets allocated to the trustedFillers.

Customer Response: Acknowledged. The proposed circuit breaker was implemented in acdd1 but subsequently reverted. The team decided to retain the trusted filler trust assumption, as protecting against a malicious or compromised trusted filler would introduce additional

complexity and potential failure modes. The associated risk is acknowledged as a limitation of the protocol.

I-05: AUCTION_LAUNCHER can bypass the weight.spot=0 restriction on the REBALANCE_MANAGER

Files:

contracts/Folio.sol contracts/utils/RebalancingLib.sol

Description:

When weight control is available, there is a restriction on weights so that spot cannot be 0 while high>0 (RebalanceingLib.startRebalance()). This restriction exists since weight.spot=0 allows removing an asset from the basket, but also requires the balanceOf() to be 0, which we cannot reach if weight.high>0. There is no such restriction when starting an auction - so if weight.low=0, weight.high>0, then the AUCTION_LAUNCHER can set the weight.spot to 0.

Recommendations:

Consider changing the restriction in removeFromBasket() to use weight.high, or requiring weight.low>0 (instead of spot) if weight.high>0.

Alternatively, enforce this restriction also in RebalanceingLib.openAuction().

Customer Response: Fixed in 57ca0. This guard was there to prevent a difficult griefing case of removeFromBasket(). We've switched (57ca0) to a model that passively removes from basket after trusted fills instead.

Fix Review: Fixed.

I-06: If no shares remain, mint() cannot be called

Files:
contracts/Folio.sol

Description:
When calculating the amount of assets expected on a mint(), by calling _toAssets(), we divide the new shares by the current shares (Folio.sol line 902). When the total shares are 0, this will revert. This is possible only if the last user redeemed their last shares (including the creator immediately after creation).

Recommendations:
Consider explicitly deprecating the folio when the last shares are redeemed.

Customer Response: Acknowledged.

I-07: addToBasket() and removeFromBasket() may grief redeem() and mint() calls

Files:
contracts/Folio.sol

Description:
When redeeming shares, a precise list of underlying assets must be passed. When minting shares, allowance to relevant assets must be given. If addToBasket() or removeFromBasket() front run a call to redeem(), it will revert, and if addToBasket() front runs mint(), it will revert.

Recommendations:
Consider documenting this risk.

Customer Response: Acknowledged.

I-08: Closing trusted filler does not automatically remove assets from basket

Files:
contracts/Folio.sol contracts/utils/RebalancingLib.sol

Description:
When bid() takes the last remaining balance of an asset, that asset is automatically removed from the basket. This logic is missing in the close trusted filler flow, allowing empty assets to remain in the basket. While the permissionless removeFromBasket() option mitigates this, the logic can be added to the close trusted folio flow, preventing this situation in the first place.

Recommendations:
Add a 0 balance check to the sold asset when closing a trusted folio, and remove the asset from the basket when the balance is 0.

Customer Response: Fixed in 57ca0. We've switched to a model that passively removes from basket after a trusted fill removes the last tokens, as with regular bids: 57ca0.

Fix Review: Fixed.

I-09: Setting the folio as a fee recipient DOSs fee distribution

Files:
contracts/Folio.sol

Description:
The folio blocks minting and transfer of shares to itself. If it is set as one of the fee recipients, or as the DAO recipient, and any amount of fees accumulates, distributeFees() will revert. distributeFees() is called when changing the fee recipients, so this situation will block fee distribution permanently.

Recommendations:
Add a check when setting fee recipients (and the DAO recipient) that none of them are the folio itself.

Customer Response: Fixed in ebef7. Mitigated to prevent setting Folio as fee recipient, and skip over when DAO has set Folio as fee recipient: ebef7.

Fix Review: Fixed.

I-10: Trusted fill flow can break the rebalance.details[].maxAuctionSize cap

Files:
contracts/Folio.sol
node_modules/@reserve-protocol/trusted-fillers/contracts/fillers/cowswap/CowSwapFiller.sol

Description:
In the trusted fill flow, the auction price is initially computed directly via `_price()` inside `getBid()`, and this value is used to determine the required amount of `buyToken`. The trusted filler later recomputes the effective price using the finalized sold/bought token amounts. This introduces an additional rounding step compared to the normal `bid()` flow.
Concretely, the amount of `buyToken` is already rounded up during the initial calculation, and the subsequent price recomputation in the trusted fill flow may round up again. As a result, the trusted fill path can require slightly more `buyToken` than the standard `bid()` path for the same trade.
The `maxAuctionSize` cap is derived assuming the standard `bid()` calculation flow. Due to the extra rounding in the trusted fill path, the final required `buyToken` amount can exceed the cap by a small rounding margin.

Recommendation: This behavior can only increase the amount of `buyToken` received by the folio. It cannot cause the folio to sell more tokens than intended or exceed the intended sell-side exposure. In practice, this acts only as a small additional donation route to the folio caused by conservative rounding.
While there are no negative side effects for users, this should be documented as an additional vector for donations, and should be taken into account if implementing the suggestions in I-02 on virtual balances.

Customer Response: Acknowledged.

Formal Verification

Verification Methodology

We performed verification of the **Reserve** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and **Reserve's** implementation source code written in Solidity.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

1. We assume there are three tokens in the basket (see assumeThreeTokens function in the spec file) to make the verification tasks tractable for the prover.
2. We assume that there is no TVL fee calculation or distribution (by assuming that `block.timestamp - lastPoke() < 86400;` and that distributeFees() is a no-op).
3. For the trustedFill, we implement a mock simulating the behaviour in certora/mocks/MockTrustedFiller.sol by closing the filler via resetting its values. Since RebalancingLib.closeTrustedFiller first closes the filler and only then checks the activeTrustedFill.sellAmount, we needed to comment out resetting the sellAmount value for the rule closeTrustedFillUpdatesTradedAmountsCorrectly.
4. For the rules tradedNeverExceedsAuctionSize, maxAuctionSizeIsNotExceeded, notInAuctionShareRatioDoesNotDecrease and onlyTradeTokensOutsideRange, we assume that the trusted fill flow is not enabled. The first two rules are covered separately for trusted fills. The third rule assumes no token exchange via bid either. The last one relies on the getBid functionality which is identical for both flows.
5. For all rules but the priceMonotonicity, we replace the _price function with _priceSimplified where we substitute the exponential decay computation by a deterministic but random function due to intractability. This is a sound overapproximation, thus safely enhancing the performance of the prover.

6. We replace some of the math functions (max, min, average, mulDiv) in OpenZeppelin library by CVL implementations that are more efficient for the prover. These implementations have been shown correct independently of this project.
7. We model the ERC20 functionality in CVL.
8. For priceMonotonicity, we assume that `MathLib.ln(x) < 10^19` for all `x < 2 * 10^22`. Since the fixed-point representation uses `10^18` to represent `1`, this corresponds to assuming that `ln(x) < 10` for all real values `x < 20,000`. This bound is comfortably above the actual mathematical value, since `ln(20,000)` is approximately `9.9`. The assumption is necessary so the cast `int256(k * elapsed)` does not overflow, since overflow would invalidate the monotonicity argument independently of the logarithm/exponential behavior.
9. For priceMonotonicity, we additionally assume that `MathLib.exp` is monotonic over the range of inputs reachable in the protocol flow. This assumption is necessary because the underlying helper `Common.exp2` is known not to be globally monotonic: there exist isolated pairs of very closely spaced inputs for which `exp2(a) < exp2(b)` even when `a > b`. However, the known counterexamples require input values that are extremely close together. In the context of this project, the values passed into `MathLib.exp` are constrained by the surrounding computations and input granularity, and do not appear to allow such closely spaced problematic pairs. No counterexample was found by the prover or by automated testing within the reachable state space. Nevertheless, this does not constitute a formal proof that `MathLib.exp` is monotonic in all reachable executions; rather, it is a library-level assumption relied upon by this rule.
10. We introduce a Harness contract that inherits from the original Folio and extends it by exposing functions that help us to formulate rules for the formal verification. Particularly, we change the visibility modifiers of basket, activeTrustedFill and rebalance from private to internal so they are accessible in the CVL spec.

Formal Verification Properties Overview

ID	Title	Status
P-01	When not in an auction, token/share ratio should not decrease, for each underlying token, up to fees	Verified
P-02	When in auction, the total value of each share should not decrease.	Verified
P-03	For each token, in each auction, the auction limit (rebalance.details[].maxAuctionSize) is not exceeded	Violated
P-04	Bid flow is equivalent to trusted fill flow	Verified
P-05	When auction is opened, we only trade tokens outside the auction's range	Verified
P-06	_price() monotonically decreases throughout an auction	Verified
P-07	Splitting bids is equivalent to a large bid, up to rounding	Verified
P-08	Share quantities remain the same unless mint or redeem are called or fees are recalculated/distributed (e.g. during totalSupply())	Verified
P-09	We can sell tokens via bid if and only if they are in surplus (above weight.high). We can buy tokens via bid if and only if they are in deficit	Verified

	(below weight.low). We cannot create deficit nor surplus via bid.	
P-10	If a token is removed from the basket, the current balance of it is 0, or the user is admin.	Verified

Detailed Properties

P-01. When not in an auction, token/share ratio should not decrease, for each underlying token, up to fees

Status: Verified	**High-level description:** Prove that shares do not lose value, even when looking at each asset separately, except for the edge cases where we can give tokens (bid, trustedFills), and fees (that dilute share value).

Rule Name	Status	Description	Link to rule report
notInAuctionShareRatioDoesNotDecrease	Verified	*Validates that for an arbitrary asset in the basket, the amount of this asset divided by total shares, does not decrease across calls to non-bid, non-filler and non-mint functions.*	*report*
shareRatioDoesNotDecreaseOnMintToken 1 (2 and 3)	Verified	*Validates that for a given asset in the basket, the asset to share ratio does not decrease when mint is called.*	*report*

P-02. When in auction, the total value of each share should not decrease.

Status: Verified	**High-level description:** Prove that shares do not lose value, except for fees. We omit trustedFill flow. We assume that the price for tokens in surplus is the low price (the price we are still willing to sell the token for) and the price for the tokens in deficit is the high price (the price we are still willing to buy the token for). The price to evaluate the other tokens does not matter since the token to share ratio does not decrease for such tokens. The property is split into several rules in order to make the verification possible. First, we split the property into several rules based on which function might be called: - *totalValueOfTokenDoesNotDecrease* - here we prove the property for all functions except mint and bid - *totalValueOfTokenDoesNotDecreaseOnMint* - here we prove the property for the mint function, we assume that the token to share ratio does not decrease when mint is called - *shareRatioDoesNotDecreaseOnMintToken* - this rule proves the assumption the previous rule

	made, i.e., the token to share ratio does not decrease when mint is called. To avoid prover performance issues, we had to create three separate rules for this property to cover different tokens that might be in the basket (we assume there are three tokens only as discussed in the General Assumptions section) now it remains to prove the property for the bid, we need to split the proof even further into several simpler rules: - *tokenValuesAfterBid* rule proves that the sellToken amount decreases by sellAmount (the amount of sellToken the Folio gives out) while buyToken amount increases by sellAmount * price / D27 (rounded up) where price is calculated in RebalancingLib._price - *tokenRanges* rule proves that the sellToken is in surplus and buyToken is in deficit before the bid is called (otherwise the bid would revert) which justifies using the low price for the sellToken and the high price for the buyToken - *tokenPrices* this rule proves that the price given by the RebalancingLib._price function can be bounded from above by sellPrice * D27 / buyPrice, where sellPrice is the low price of the sellToken and buyPrice is the high price of the buyToken - *valueCalculation* rule uses the previous results to show that the total value of the Folio cannot decrease. Using the property P-08, we know that the share amount does not change when bid is called, therefore share value cannot decrease. This completes the proof.

Rule Name	Status	Description	Link to rule report
totalValueOfTokenDoesNotDecrease	Verified	*Validates that for an arbitrary asset in the basket, the amount of this asset divided by total shares, does not decrease across calls to non-bid, non-filler and not mint.*	*report*
totalValueOfTokenDoesNotDecreaseOnMint	Verified	*Validates that for an arbitrary asset in the basket, the value of the assets per share does not decrease. This is a consequence of another rule that states that the token to share ratio does not decrease during mint for any token.*	*report*
shareRatioDoesNotDecreaseOnMintToken1 (2 and 3)	Verified	*Validates that for the first (second, third) token in the basket, the mint does not decrease the token to share ratio.*	*report*
tokenValuesAfterBid	Verified	*Validates how tokens change after bid based on the sellAmount and price.*	*report*

tokenRanges	Verified	*Validates that if a bid was successfully called, sellToken had to be in surplus and butToken had to be in deficit.*	*report*
tokenPrices	Verified	*Proves that the price is capped from below by sellPrice*D27/buyPrice where sellPrice is the low price of the sellToken in the auction and the buyPrice is the high price of the buyToken in the auction.*	*report*
valueCalculation	Verified	*Given the properties in other rules, this rule validates that the portfolio value after bid does not decrease.*	*report*

P-03. For each token, in each auction, the auction limit (rebalance.details[].maxAuctionSize) is not exceeded

Status: Violated	**High-level description:** Verifies that the limits set by the REBALANCE_MANAGER for the maximum amount of each token traded each auction, is respected. Violation: The maxAuctionSize **is not exceeded** during the bid flow. This is shown in the *tradedNeverExceedsAuctionSize* invariant. We also show in the *AuctionSizeIsNotExceeded* rule that the traded value for each token is updated correctly based on the token balances before and after. The property **can be violated during the trustedFill flow** because the protocol can receive more buyTokens than it would in the bid flow. The rule closeTrustedFillUpdatesTradedAmountsCorrectly shows that the bookkeeping of the traded amounts is correct for the trustedFill flow. The rule *fillersCannotExceedTheTradedCapSimple* shows the violation using a single exchange with the trustedFiller.

Rule Name	Status	Description	Link to rule report
maxAuctionSizeIsNotExceeded	Verified	*Verifies that the difference in a token balance matches the difference in auction.traded, and that the new value for auction.traded is less than the limit. The exceptions are: mint, redeem, transfers (i.e. donations), and createTrustedFill.*	*report*
tradedNeverExceedsAuctionSize	Verified	*Verifies that the auctions.traded[token] <= rebalance.details[token].maxAuctionSize except for the trusted fill flow*	*report*

closeTrustedFillUpdatesTradedAmountsCorrectly	Verified	*verifies that the traded amounts are correctly updated when the filler is closed - small change is needed for the mock, upon close, the mock zeroes out the sellAmount, we have to avoid it for this rule*	*report*
fillersCannotExceedTheTradedCap	Violated	*Checks whether the trusted filler flow can break the traded cap given by the rebalance manager. This rule is violated due to a different way of calculating the bid amount for the fill flow which results in different rounding. This does not pose a problem as discussed in informational issue I-10*	*report*
fillersCannotExceedTheTradedCapSimple	Violated	*Checks whether the trusted filler flow can break the traded cap using only one exchange with the filler. This rule is violated due to a different way of calculating the bid amount for the fill flow which results in different rounding. This does not pose a problem as discussed in informational issue I-10*	*report*

P-04. Bid flow is equivalent to trusted fill flow

Status: Verified

High-level description:
Checks a direct bid() call compared to opening a trustedFill (implemented by a mock contract), and verifies that these flows are equivalent.

The equivalence:
There are subtle differences as discussed in I-4, I-8 and I-10. Given the mock implementation, we show that the protocol is never worse when using trusted fill flow, i.e., it sends out the same amount of sellToken and receives at least the amount of buyToken as it would from the bid flow. We also provide an estimate on how many extra buyTokens it can receive. This depends on the mock where the computation follows the minimum buyToken amounts the CowSwapFiller requires to allow the trade.

Rule Name	Status	Description	Link to rule report
fillerIsNeverWorse	Verified	*Trades via bid() and via a trustedFill, both from the same starting state, validates that the trustedFill path results in at least the same underlying balance as the bid() flow. This rule uses the trustedFill mock.*	*report*
otherTokensAreTheSameWithFills	Verified	*Trading via bid and trusted fills gives the same result for sellToken and other unrelated tokens.*	*report*

buyTokenDifferenceCap	Verified	*Trades via bid and trustedFill, both from the same starting state. Proves a bound on the difference in buyToken. This rule uses the trustedFill mock.*	*report*
buyCalculationFillFlow	Verified	*Helper rule proving the bound for the buyToken amount.*	*report*

P-05. When auction is opened, we only trade tokens outside the auction's range

Status: Verified	**High-level description:** We should only be able to sell tokens in surplus, and buy tokens in deficit, relative to the weight and limits.

Rule Name	Status	Description	Link to rule report
onlyTradeTokensOutsideRange	Verified	*Verifies connections between the balanceOf() a token increasing/decreasing, and between whether the token is in surplus or deficit, across calls to bid().* *Here specifically, we need the assertion "balanceBefore != balanceAfter => (canSell || canBuy)", meaning: if we are ending up with a different balanceOf() for a token, it was either in surplus or deficit.*	*report*

P-06. _price() monotonically decreases throughout an auction

Status: Verified	**High-level description:** Verifies that within each auction, the price decreases monotonically. Assumes the library function MathLib.exp is monotonic and that the MathLib.ln function returns at most 10^19 (i.e. 10 in the 18 decimal representation) for all inputs below 2*10^22 (i.e. 20000). **Safety of the assumptions:** The assumption on the ln function is only to ensure that the cast int256(k * elapsed) does not overflow and can be relaxed. As we stated in the General Assumptions section, we **do not prove** that the assumptions we use for the exp function hold. The MathLib.exp calls Math.exp which calls Math.exp2 and that one finally calls Common.exp2. We can prove that if the Common.exp2 was monotonic for all admissible inputs, the whole MathLib.exp would be as well. However, the function Common.exp2 is not monotonic by itself. On the other hand, all the counterexamples we found, e.g. exp2(350488137400481 4807056) <

	exp2(350488137400481480705 5), require the input values to be off by one. In practice, this does not happen. Math.sol line 229 alone forces the smallest non-zero difference in the input values to the Common.exp2 to be at least 18 apart. Restrictions on the values in the whole _price -> Math.exp -> Math.exp2 -> Common.exp2 flow can only make this input difference larger.

Rule Name	Status	Description	Link to rule report
priceMonotonicity	Verified	*Verifies that for any two times within the same auction, the price at the earlier time is higher. Assumes the library function MathLib.exp is monotonic and that the MathLib.ln function returns at most 10^19 (i.e. 10 in the 18 decimal representation) for all inputs below 2*10^22 (i.e. 20000).*	report
priceMonotonicityAtStart	Verified	*shows that the calculation using the ln and exp functions gives the startPrice at startTime*	report

P-07. Splitting bids is equivalent to a large bid, up to rounding

Status: Verified	**High-level description:** Verifies that splitting bids results in the same underlying token balance as placing a single bid for the sum of their values, up to rounding.

Rule Name	Status	Description	Link to rule report
bidResult	Verified	*Verifies that calling a bid with sellAmount will decrease the balance of sellToken by this amount while increasing the balance of buy token by the value returned from the bid function. Also shows that the returned value is sellAmount * price / D27 rounded up.*	report
splitCalculation	Verified	*Verifies that one bid is always better than two smaller bids for the Folio while the difference between one and two bids is at most 1 wei. We do not call the bid function explicitly but use the result from the bidResult rule.*	report

P-08. Share quantities remain the same unless mint or redeem are called or fees are recalculated/distributed (e.g. during totalSupply())

Status: Verified	**High-level description:** The total number of shares has 3 defined ways to change: TVL fees, mint and redeem. This property verifies that no other paths exist. We exclude the trusted fill flow.

Rule Name	Status	Description	Link to rule report
onlyMintRedeemAndFeesCanChangeShares	Verified	*Assuming no TVL fee recalculation, and that the called function is not mint, redeem, or createTrustedFill, the totalSupply of shares remains the same.*	report

P-09. We can sell tokens via bid if and only if they are in surplus (above weight.high). We can buy tokens via bid if and only if they are in deficit (below weight.low). We cannot create deficit nor surplus via bid.

Status: Verified	**High-level description:** When trading tokens in an auction, the tokens can be in a surplus, deficit or within auction range based on the weights, limits, token balance and share amount. We prove that trading a particular token is allowed only in situations when it should be allowed. We focus on the bid function.

Rule Name	Status	Description	Link to rule report
onlyTradeTokensOutsideRange	Verified	*Verifies connections between the balanceOf() a token increasing/decreasing, and between whether the token is in surplus or deficit, across calls to bid().* *For the sell part, we need* *canSell => balanceBefore >= balanceAfter;* *balanceBefore > balanceAfter => canSell;* *for the buy part:* *canBuy => balanceBefore <= balanceAfter;* *balanceBefore < balanceAfter => canBuy;* *and for the last part is stated as* *!canSell => !isTokenInSurplus(e,token);* *!canBuy => !isTokenInDeficit(e,token);*	report

P-10. If a token is removed from the basket, the current balance of it is 0, or the user is admin.

Status: Verified	**High-level description:** We prove that situations that allow a token to be removed from a basket are only those intended: calling removeFromBasket when the msg.sender is admin or the balance and weight of the token are 0 removing the last token via bid

Rule Name	Status	Description	Link to rule report
removeTokenFromBasket	Verified	*Verifies that if a token was in basket but is no longer, either admin called removeFromBasket or the token balance is 0*	*report*

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.